



# การสร้าง Dashboard

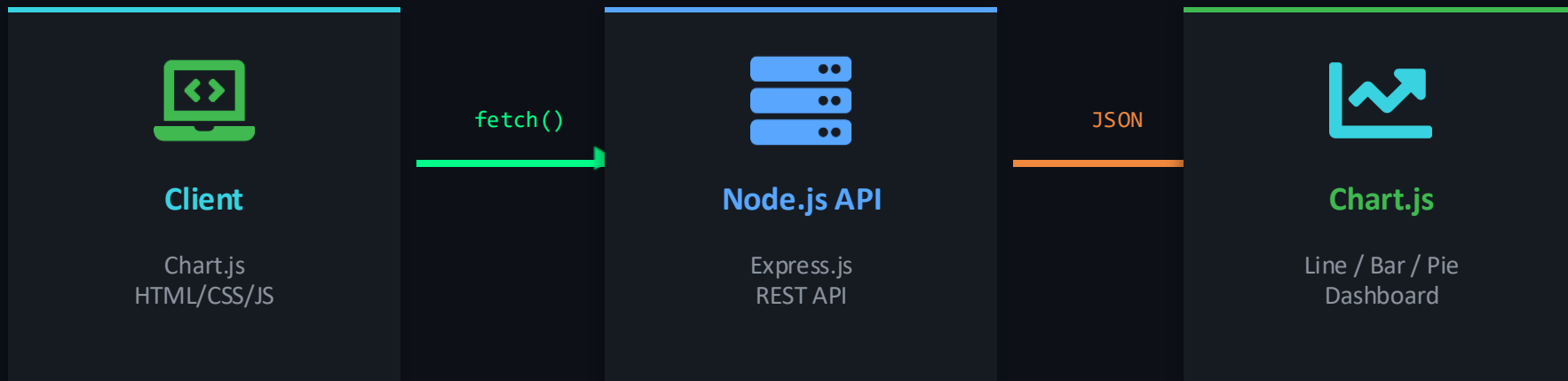
ด้วย Chart.js + Node.js API

Advanced + Production

สำหรับ Frontend / Fullstack Developer

# Overview Architecture

ภาพรวมสถาปัตยกรรม: Frontend ↔ Backend ↔ Data



Flow: Client → API → JSON → Chart

- ▶ Frontend ส่ง request ไปยัง API ผ่าน `fetch()`
- ▶ Backend ประมวลผลและส่ง JSON response กลับ

# ตัวอย่าง API (Node.js)



Express route: /api/sales — return JSON data



```
// สร้าง API endpoint สำหรับข้อมูลยอดขาย
app.get("/api/sales", (req, res) => {
  res.json([
    { date: "2026-01", total: 12000 },
    { date: "2026-02", total: 18000 },
    { date: "2026-03", total: 15000 },
    { date: "2026-04", total: 22000 }
  ])
})
```

- ▶ ใช้ Express.js สร้าง REST API
- ▶ Endpoint: GET /api/sales
- ▶ Response เป็น JSON Array
- ▶ แต่ละ object มี date และ total
- ▶ พร้อมเชื่อมต่อกับ Frontend

- ▶ สามารถเชื่อมต่อ Database จริงได้ (MongoDB, MySQL)
- ▶ เพิ่ม middleware สำหรับ authentication ได้

# Fetch API จาก Frontend



ใช้ fetch() + async/await ดึงข้อมูลจาก API



```
// ดึงข้อมูลจาก API ด้วย async/await
async function loadSalesData() {
  const res = await fetch("/api/sales")
  const data = await res.json()

  console.log(data)
  // [{ date: "2026-01", total: 12000 }, ...]

  return data
}
```

## async/await

เขียนโค้ด asynchronous แบบอ่านง่าย

## fetch()

Web API สำหรับ HTTP request

## .json()

แปลง response เป็น JavaScript object

- ▶ ใช้ async/await แทน .then() chain เพื่อความอ่านง่าย
- ▶ ควรเพิ่ม try/catch สำหรับ error handling ในงานจริง



# Mapping Data → Chart

แปลง JSON data เป็น labels และ datasets สำหรับ Chart.js



```
// แปลงข้อมูล JSON เป็นรูปแบบที่ Chart.js ต้องการ
const labels = data.map(d => d.date)
// ["2026-01", "2026-02", "2026-03", ...]

const values = data.map(d => d.total)
// [12000, 18000, 15000, ...]
```

## Data Flow

JSON Array

↓ .map()

labels[] + values[]

↓

Chart.js Dataset

- ▶ labels = แกน X ของกราฟ (วันที่, เดือน, หมวดหมู่)
- ▶ values = แกน Y ของกราฟ (ตัวเลข, จำนวน, ยอดเงิน)
- ▶ Array.map() เป็นวิธีมาตรฐานในการ transform ข้อมูล

# สร้าง Chart พื้นฐาน



สร้าง Line Chart จากข้อมูลที่ map แล้ว

```
● ● ●  
// สร้าง Line Chart พื้นฐาน  
new Chart(ctx, {  
  type: "line",  
  data: {  
    labels,           // แกน X  
    datasets: [{  
      label: "Sales",  
      data: values,   // แกน Y  
      borderColor: "#58A6FF",  
      tension: 0.4  
    }]  
  }  
})
```

## โครงสร้างหลัก

type — ประเภทกราฟ

data.labels — ค่าแกน X

data.datasets — ชุดข้อมูล

borderColor — สีเส้นกราฟ

tension — ความโค้งของเส้น



# เจาะลึกโครงสร้าง Chart.js

3 ส่วนหลัก: type, data, options



## type

ประเภทกราฟ  
line, bar, pie,  
doughnut, radar,  
scatter, bubble



## data

ข้อมูลกราฟ  
labels[]  
datasets[]  
แต่ละ dataset  
มี data, label, color



## options

การปรับแต่ง  
plugins, scales,  
animation, interaction  
★ สำคัญที่สุด!

★ options คือหัวใจของการ customize — จะเจาะลึกในสไลด์ถัดไป

# Options เบื้องต้น



การตั้งค่าพื้นฐานของ Chart.js

## responsive

true

ปรับขนาดตาม container อัตโนมัติ

## maintainAspectRatio

false

ให้ chart เต็มพื้นที่ container

## parsing

{ xAxisKey, yAxisKey }

กำหนด key ในข้อมูลสำหรับแต่ละแกน

## normalized

true

เพิ่ม performance เมื่อข้อมูลเรียงแล้ว



# Plugins: legend, title, tooltip



ปรับแต่ง UI elements ของกราฟ



```
// ตั้งค่า plugins สำหรับ UI
options: {
  plugins: {
    legend: {
      display: true,
      position: "top"
    },
    title: {
      display: true,
      text: "Sales Report",
      color: "#E6EDF3"
    },
    tooltip: {
      enabled: true,
      mode: "index"
    }
  }
}
```

## legend

แสดงคำอธิบายสี/ชื่อ dataset

## title

ชื่อกราฟด้านบน

## tooltip

แสดงรายละเอียดเมื่อ hover

# Scales (แกน X/Y)



ควบคุมแกน, grid lines, และ ticks



// ตั้งค่าแกน X และ Y

```
scales: {  
  y: {  
    beginAtZero: true,  
    ticks: {  
      color: "#8B949E",  
      callback: (v) => v.toLocaleString() + " ฿"  
    },  
    grid: {  
      color: "rgba(139,148,158,0.15)"  
    }  
  },  
  x: {  
    ticks: { color: "#8B949E" },  
    grid: { display: false }  
  }  
}
```

## **beginAtZero**

เริ่มแกน Y จาก 0

## **ticks**

ปรับสี, format ตัวเลข, font

## **grid**

เส้น grid สีอ่อน / ซ่อน

## **callback**

format ค่าที่แสดง (เช่น + สัญลักษณ์เงิน)

# Interaction



ควบคุมพฤติกรรมเมื่อผู้ใช้ hover และ click

**mode: 'index'**

แสดง tooltip ทุก dataset ในแกน X เดียวกัน

**mode: 'nearest'**

แสดง tooltip จุดที่ใกล้เคอร์เซอร์ที่สุด

**intersect: false**

ไม่ต้อง hover ตรงจุดก็แสดง tooltip ได้

**onHover / onClick**

callback function สำหรับ custom behavior

*Tip: ใช้ mode: 'index' + intersect: false สำหรับ dashboard ที่ใช้งานง่าย*

# Animation



สร้างความน่าสนใจด้วย animation ที่ลื่นไหล

```
● ● ●  
// ตั้งค่า animation ให้กราฟ  
animation: {  
  duration: 1500,      // ระยะเวลา (ms)  
  easing: "easeOutQuart", // รูปแบบการเคลื่อนไหว  
  delay: (ctx) => {  
    return ctx.dataIndex * 100 // หน่วงทีละจุด  
  }  
}
```

## Easing Options

linear — เท่ากันตลอด

easeOutQuart — ช้าลงตอนจบ

easeInOutCubic — นุ่มนวล

easeOutBounce — เด้งตอนจบ

- ▶ delay function ทำให้แต่ละจุดปรากฏทีละจุด (stagger effect)
- ▶ ใช้ animation ไม่มากเกินไปเพื่อไม่ให้รบกวนผู้ใช้



# Elements: Line & Point

ปรับแต่งรูปร่างของเส้นและจุดบนกราฟ



```
// ปรับแต่ง elements
elements: {
  line: {
    tension: 0.4,          // ความโค้ง (0 = ตรง, 1 = โค้งมาก)
    borderWidth: 2,        // ความหนาเส้น
    borderCapStyle: "round"
  },
  point: {
    radius: 4,             // ขนาดจุด
    hoverRadius: 8,        // ขนาดจุดตอน hover
    backgroundColor: "#58A6FF"
  }
}
```

## Line Properties

tension — ค่า 0.3-0.4 สายที่สุด

borderWidth — แนะนำ 2-3px

## Point Properties

radius — ปกติ 3-5px

hoverRadius — ขยายตอน hover



## UI/UX: สีและ Gradient



```
// สร้าง gradient สำหรับพื้นที่ใต้กราฟ
const gradient = ctx.createLinearGradient(0, 0, 0, 400)
gradient.addColorStop(0, "rgba(88,166,255,0.4)")
gradient.addColorStop(1, "rgba(88,166,255,0.0)")

// ใส่ใน dataset
backgroundColor: gradient
```

- ▶ Gradient ทำให้กราฟดูมีมิติ
- ▶ ใช้ rgba สำหรับความโปร่งใส

# Smooth Line



ปรับเส้นกราฟให้โค้งมน ดูเป็นมืออาชีพ

**tension: 0**

เส้นตรง แข็ง



**tension: 0.4**

เส้นโค้ง สวยงาม

- ▶ ค่า tension 0.3-0.4 คือค่าที่เหมาะสมที่สุดสำหรับ dashboard
- ▶ ค่ามากกว่า 0.5 อาจทำให้กราฟบิดเบี้ยวจากข้อมูลจริง
- ▶ เส้นโค้งช่วยให้ผู้ชมมองเห็น trend ได้ง่ายขึ้น



# Tooltip แบบสวย

Custom label และ format ตัวเลขใน tooltip



```
// ปรับแต่ง tooltip ให้สวย
tooltip: {
  backgroundColor: "rgba(22,27,34,0.95)",
  titleColor: "#58A6FF",
  bodyColor: "#E6EDF3",
  borderColor: "#58A6FF",
  borderWidth: 1,
  cornerRadius: 8,
  padding: 12,
  callbacks: {
    label: (ctx) => {
      // format ตัวเลขแบบไทย
      const val = ctx.parsed.y.toLocaleString()
      return `ยอดขาย: ${val} บาท`
    }
  }
}
```

## Tooltip Styling

backgroundColor — สีพื้น

borderColor — ขอบ tooltip

cornerRadius — มุมโค้ง

callbacks.label — format ข้อความ

*Tip: ใช้ toLocaleString()*

*format ตัวเลขให้อ่านง่าย*



# Legend แบบมืออาชีพ



ปรับตำแหน่งและสไตล์ของ Legend



```
// ปรับแต่ง Legend
legend: {
  position: "top",          // top, bottom, left, right
  align: "end",            // start, center, end
  labels: {
    color: "#E6EDF3",
    usePointStyle: true, // ใช้วงกลมแทนสีเหลี่ยม
    pointStyle: "circle",
    padding: 20,
    font: {
      size: 13,
      family: "Calibri"
    }
  }
}
```

- ▶ usePointStyle ทำให้ legend ดูทันสมัย
- ▶ position: 'top' เหมาะกับ dashboard
- ▶ ปรับ padding เพิ่มระยะห่าง
- ▶ ใช้สีที่อ่านง่ายบนพื้นมืด



# Real-time Update

อัปเดตกราฟแบบ real-time ด้วย setInterval



```
// อัปเดตข้อมูลแบบ real-time
async function updateChart() {
  const res = await fetch("/api/sales/latest")
  const newData = await res.json()

  // เพิ่มข้อมูลใหม่
  chart.data.labels.push(newData.date)
  chart.data.datasets[0].data.push(newData.total)

  // ลบข้อมูลเก่า (เก็บแค่ 12 จุด)
  if (chart.data.labels.length > 12) {
    chart.data.labels.shift()
    chart.data.datasets[0].data.shift()
  }

  chart.update("none") // ไม่ใส่ animation
}
```

```
setInterval(updateChart, 5000) // ทุก 5 วินาที
```

## Key Concepts

### `chart.update()`

บอก Chart.js ให้ re-render

### "none" mode

ข้าม animation เพื่อ performance

### `.shift() + .push()`

เลื่อนข้อมูลแบบ sliding window



# Multiple Datasets

เปรียบเทียบหลายชุดข้อมูลในกราฟเดียว



// หลาย datasets ในกราฟเดียว

```
datasets: [  
  {  
    label: "Sales",  
    data: salesData,  
    borderColor: "#58A6FF",  
    backgroundColor: gradient1  
  },  
  {  
    label: "Expenses",  
    data: expenseData,  
    borderColor: "#F778BA",  
    backgroundColor: gradient2  
  },  
  {  
    label: "Profit",  
    data: profitData,  
    borderColor: "#3FB950",  
    backgroundColor: gradient3  
  }  
]
```

- ▶ แต่ละ dataset มีสี/สไตล์แยก
- ▶ ใช้สีที่ต่างกันชัดเจน
- ▶ Legend ช่วยแยก dataset
- ▶ ไม่ควรเกิน 4-5 datasets
- ▶ ใช้ interaction mode: 'index'

# Performance Optimization



ทำให้กราฟรันเร็วแม้ข้อมูลเยอะ

**animation: false**

ปิด animation เพิ่มความเร็วเมื่อข้อมูลมาก

**decimation plugin**

ลดจำนวนจุดที่แสดง (เช่น 10,000 → 500)

**normalized: true**

บอก Chart.js ว่าข้อมูลเรียงแล้ว ไม่ต้อง sort

**spanGaps: true**

ข้ามค่า null ไม่ต้องตัดเส้น ลด rendering

**parsing: false**

ส่งข้อมูลในรูปแบบที่ Chart.js ต้องการโดยตรง

# Responsive Dashboard Layout



จัดวาง dashboard ด้วย CSS Grid / Flexbox



```
/* CSS Grid สำหรับ Dashboard */
.dashboard {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 1.5rem;
  padding: 1.5rem;
}

/* Responsive: 1 column บนมือถือ */
@media (max-width: 768px) {
  .dashboard {
    grid-template-columns: 1fr;
  }
}

.chart-card {
  background: #161B22;
  border-radius: 12px;
  padding: 1.5rem;
}
```

## Layout Tips

CSS Grid — จัดวาง 2-4 คอลัมน์

gap — ระยะห่างระหว่าง chart

responsive: true ใน Chart.js

Media query ปรับ mobile

*Tip: ใช้ min-height กัน  
chart หดเล็กเกินไป*



# Error Handling

จัดการ error จาก API อย่างมืออาชีพ



```
// จัดการ error แบบครบวงจร
async function loadChart() {
  try {
    const res = await fetch("/api/sales")

    if (!res.ok) {
      throw new Error(`API Error: ${res.status}`)
    }

    const data = await res.json()
    renderChart(data)
  } catch (error) {
    console.error("Failed:", error)

    // แสดงข้อความ error บน UI
    showErrorMessage("ไม่สามารถโหลดข้อมูลได้")

    // ใช้ fallback data (ข้อมูลสำรอง)
    renderChart(FALLBACK_DATA)
  }
}
```

## Error Types

### Network Error

เซิร์ฟเวอร์ไม่ตอบ, timeout

### HTTP Error (4xx, 5xx)

server error, unauthorized

### Parse Error

JSON ไม่ถูกต้อง

### Fallback Data

แสดงข้อมูลสำรองแทน blank

# Loading State



แสดง spinner ขณะรอข้อมูลจาก API

```
● ● ●  
// จัดการ loading state  
function showLoading(containerId) {  
  const el = document.getElementById(containerId)  
  el.innerHTML = `  
    <div class="loading-spinner">  
      <div class="spinner"></div>  
      <p>กำลังโหลดข้อมูล...</p>  
    </div>  
  `;  
}  
  
// เรียกใช้  
showLoading("chart-container")  
const data = await fetchData()  
renderChart(data) // แทนที่ spinner ด้วย chart
```

```
● ● ●  
/* CSS Spinner */  
.spinner {  
  width: 40px; height: 40px;  
  border: 3px solid #161B22;  
  border-top: 3px solid #58A6FF;  
  border-radius: 50%;  
  animation: spin 0.8s linear infinite;  
}
```

- ▶ UX ที่ดี: แสดง loading indicator ทุกครั้งที่รอ API
- ▶ ใช้ skeleton screen สำหรับ dashboard ที่ซับซ้อน

# Best Practices



แนวทางการเขียนโค้ด dashboard ที่ดี



```
// แยก config เป็น reusable function
function createChartConfig(type, data, opts) {
  return {
    type,
    data,
    options: {
      responsive: true,
      maintainAspectRatio: false,
      plugins: { ...defaultPlugins, ...opts },
      scales: defaultScales,
      animation: defaultAnimation,
    }
  }
}

// เรียกใช้งาน
const config = createChartConfig("line", salesData)
new Chart(ctx, config)
```

## Best Practices

- ✓ แยก config จาก data
- ✓ สร้าง reusable function
- ✓ default options + override
- ✓ แยกไฟล์ chart config
- ✓ ใช้ constants สำหรับสี
- ✓ destroy() ก่อนสร้าง chart ใหม่



# Dashboard ตัวอย่าง

## Sales + Users + Revenue Charts

Total Sales

**\$284,500**

+12.5%

Active Users

**1,847**

+8.3%

Revenue

**\$156,200**

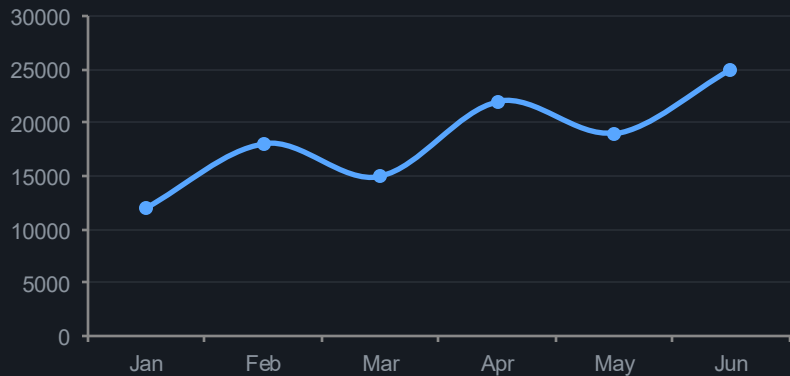
+15.7%

Conversion

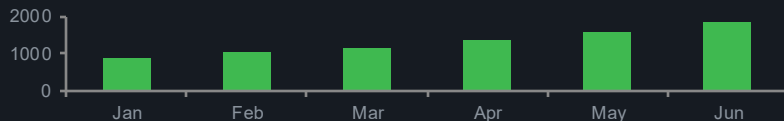
**3.24%**

+2.1%

### Sales Overview



### Active Users



### Revenue Breakdown



■ Product

■ Service



## สรุป

### Chart.js + API = Dashboard จริง

- ▶ `fetch()` ดึงข้อมูลจาก Node.js API
- ▶ `map()` แปลงข้อมูลเป็น Chart format
- ▶ `new Chart()` สร้างกราฟ + `update()`

### options = หัวใจของการ customize

- ▶ plugins — legend, title, tooltip
- ▶ scales — แกน, grid, ticks
- ▶ animation + interaction + elements



# Q & A

ถาม-ตอบ & แลกเปลี่ยนความคิดเห็น

ขอบคุณครับ!